

Mars Pharmacy ERP

Project structure, database schema, and security model — a six-step walkthrough for presenting the system.

36

tables

97

RLS policies

9

transactional RPCs

21

permissions · 4 roles

17

ordered migrations

148

tests · ~74 adversarial

Platform Next.js 16 (App Router) · React 19 · Supabase / PostgreSQL 15

Full reference docs/SCHEMA.md

Repository d:\CRM · branch master

Contents

- 1 What this is, and the four ideas** — the product, and the concepts everything else follows from
- 2 The file map and the request flow** — where each thing lives, end to end
- 3 Schema A — entities, employees, roles, permissions** — five tables, the whole access model
- 4 Schema B — products, batches, documents, ledger** — the operational heart
- 5 The security model** — five enforcement layers, all in Postgres
- 6 Presentation script** — running order, live demo, expected questions

How to use this document

Steps 1–5 are the material. Step 6 is the delivery plan: a timed running order, a five-move demo sequence, prepared answers to the hard questions, and a pre-flight checklist.

If the session is cut short, present **the four ideas (Step 1)** and **the demo (Step 6)** and drop the rest. Those two carry the argument on their own.

SOURCE OF TRUTH

Every figure in this document was read from `docs/SCHEMA.md`, which was generated from the live database, and from the source files cited throughout. Regenerate the DDL bundle with `npm run db:bundle`.

STEP 1

What this is, and the four ideas

The product in one paragraph, then the four concepts that every table, policy and function in the system follows from.

The product

Mars Pharmacy ERP — a multi-entity pharmacy ERP. Next.js 16 (App Router) on the front, Supabase / PostgreSQL 15 behind it.

“Multi-entity” is the core business framing: **one entity = one independently operated pharmacy**. Ten pharmacies share one installation and one database, but no pharmacy can see another's stock, sales, costs, or staff.

THE ONE NAMING INCONSISTENCY

The database column is `branch_id` everywhere; the UI labels it *Entity*. Remember that mapping — you will have to explain it if anyone reads the SQL alongside the screens.

Scale of what is built: **36 tables, 2 views, 97 RLS policies, 9 transactional RPCs, 17 migrations**.

The four load-bearing ideas

If the audience remembers only four things, these are the four. Everything else in the schema is a consequence of them.

1 Entity is the isolation boundary

Every entity-owned table carries `branch_id NOT NULL`. Isolation is not enforced by careful queries in application code — it is enforced by row-level security in the database. Even if the app had a bug and asked for another pharmacy's rows, Postgres returns zero rows.

2 Stock lives on batches, never on products

`products` is a catalogue — name, pack size, price. **It has no quantity column**. Physical stock lives in `product_batches`, where each row is one real lot with its own cost, its own expiry date and its own quantity.

This is non-negotiable for pharmacy: two boxes of the same medicine bought at different prices and expiring in different months are genuinely different things. Each batch also records which document created it (`source_type` / `source_id`), so any unit of stock traces back to its origin.

3 The ledger is the truth

Every single stock change appends a row to `stock_movements`, recording the `balance_after`. Nothing is ever updated or deleted there.

The invariant: **summing the ledger for a batch equals that batch's `quantity_available`** — always. That is asserted in the test suite, so it is a checked claim, not an aspiration. When someone asks “how do you know your stock numbers are right?”, this is the answer.

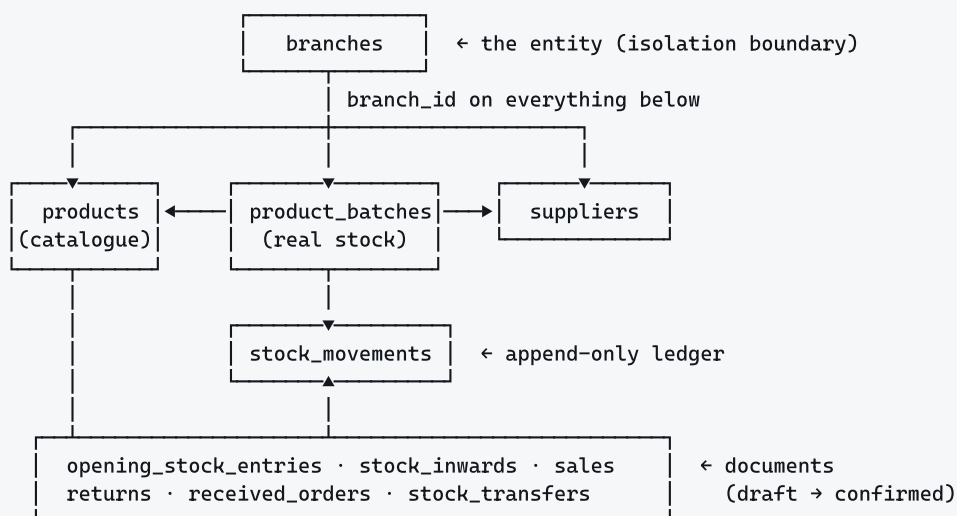
4 Documents go draft → confirmed

A stock document (opening stock, inward, sale, return) is freely editable while `status = 'draft'`.

Nothing moves stock while it is a draft.

Confirming it calls a database function that, in one transaction, creates the batches, writes the ledger rows, and locks the document. After that it is immutable. So stock only ever moves through one narrow, audited, all-or-nothing door.

How they fit together



Read it top-down: the entity owns everything. Products and suppliers are reference data. Batches are the real stock. Documents are the events that create or consume batches. The ledger records every one of those events.

STEP 2

The file map and the request flow

Three route entry points, four layers, and one function that answers “who is this and what may they see” for every screen in the application.

The shape of the app

Only **three** route entry points exist. That surprises people, so lead with it.

ROUTE	FILE	WHAT IT SERVES
/login	src/app/login/page.tsx	Sign-in
/dashboard/*	src/app/dashboard/[[...section]]/page.tsx	All 24 screens
/api/*	src/app/api/	3 routes: AI chat, import template download, report export

That `[[...section]]` is an **optional catch-all** route. `/dashboard/sales`, `/dashboard/inventory`, `/dashboard/reports` — all one file. It reads the section name from the URL and dispatches.

So there is one place where auth, permissions and entity context get resolved, instead of 24 places that each might forget.

The four layers

src/app/dashboard/views/	29 files	← what it looks like (client)
src/app/dashboard/actions/	19 files	← writes (server actions)
src/lib/data/	25 files	← reads (loaders)
src/lib/ + supabase/migrations/		← rules (shared logic + DB)

THE DISCIPLINE

Views never touch the database. Reads go through `lib/data/`, writes go through `actions/`. A view receives data as props and calls an action on submit. That is the whole rule.

The flow — reading a screen

Someone opens `/dashboard/sales-history`:

1. proxy.ts	refresh the session cookie
2. page.tsx	requireUser() who is this? → redirect to /login if nobody
3. page.tsx	permittedSections() may they see this section? → redirect if not
4. lib/data/sales.ts	getScope() which entity? what may they see?
5. supabase.from("sales")...	Postgres applies RLS
6. AppShell → sales-history-view.tsx	render

Step 4 is the one worth dwelling on. `getScope()` in `src/lib/data/scope.ts` is called by every single loader and returns one bundle:

```
{ supabase, employee, entityId, entityName, currency,
  canViewCost, canViewProfit, canViewReports, canExport }
```

So “which pharmacy am I looking at” and “am I allowed to see cost and profit” are answered once, in one function, for all 25 loaders. No loader improvises.

The flow — writing (completing a sale)

1. sales-view.tsx	calls completeSale(...)	
2. actions/sales.ts	requireUser()	
3.	permissionError(employee, "create_sales")	← gate
4.	supabase.rpc("erp_complete_sale", {...})	← one DB call
5.	revalidatePath(...) × 5	← refresh screens

Every write action is that same five-line skeleton. And note **step 4**: the action does not insert a sale row, then decrement batches, then write ledger rows. It makes **one call** to a database function that does all of it in one transaction.

That is the draft→confirmed idea from Step 1 showing up in the code — the app cannot half-complete a sale, because it does not have the ability to do the pieces separately.

The shared-rules files in `src/lib/`

These are the ones to actually open if someone asks a hard question.

FILE	WHY IT MATTERS
<code>dal.ts</code>	<i>Who is the user.</i> Loads the employee, resolves permissions, <code>requireUser()</code> / <code>requirePermission()</code> . Every request starts here.
<code>permissions.ts</code>	The 21-permission vocabulary + <code>resolvePermissions()</code> — role defaults, then per-user overrides on top
<code>entity.ts</code>	Which entity is active (cookie-driven), and who is allowed to switch
<code>data/scope.ts</code>	The per-request bundle above
<code>pricing.ts</code>	Margin / discount maths — pure, so it is unit-tested
<code>data/costs.ts</code>	Cost visibility — the <code>view_purchase_cost</code> gate

`dal.ts` and `entity.ts` are wrapped in React's `cache()` , so ten loaders on one page share **one** “who is this user” query rather than ten.

Two Next.js 16 details to have ready

Both will be asked if anyone technical is in the room.

- **`proxy.ts` at the repo root** — in Next.js 16 this file is what used to be `middleware.ts` . Same job (session refresh on every request), new name.
- **`params` and `searchParams` are Promises** — you `await` them.

Everything else

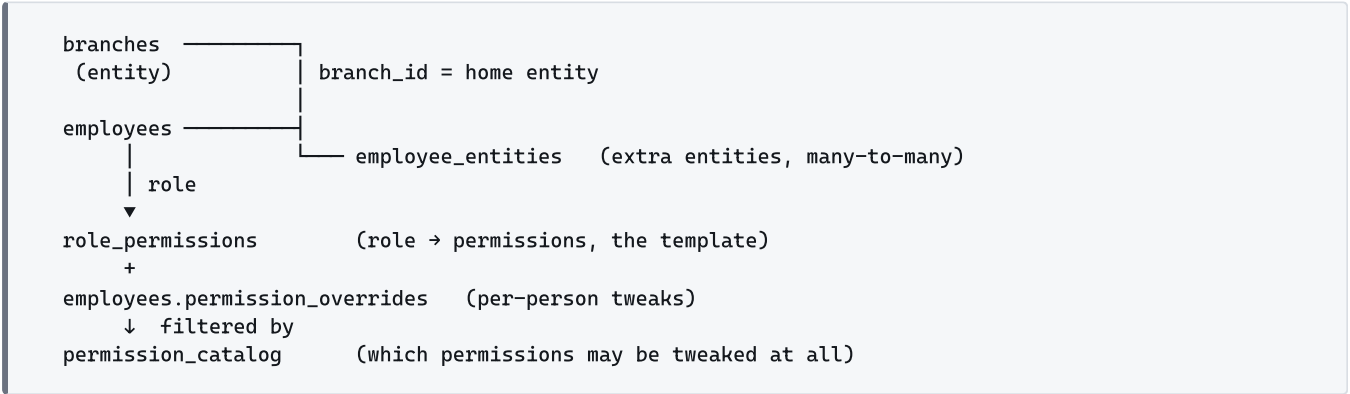
FOLDER	CONTENTS
<code>supabase/migrations/</code>	17 numbered migrations — order is load-bearing (see Step 5)
<code>tests/</code>	Unit tests + real-database RLS tests
<code>src/lib/ai/</code>	Embeddings + the Ask-AI assistant (the only part needing <code>OPENAI_API_KEY</code>)
<code>src/lib/imports/</code> , <code>src/lib/reports/</code>	Excel product import; report definitions
<code>docs/SCHEMA.md</code>	The full reference this walkthrough is drawn from

ONE-SENTENCE VERSION

One route serves every screen; reads flow through loaders that all share one scope function, writes flow through actions that each call one transactional database function, and the database enforces isolation regardless of what the app does.

Entities, employees, roles, permissions

Five tables. That is the whole access-control model — understand these and you can answer any “who can do what” question.



1. branches — the entity

One row = one pharmacy.

COLUMN	NOTES
id	uuid PK
code	UNIQUE — e.g. ENT-001
name, registered_name, phone, email, address, location, manager_name	text
currency	default TZS
timezone	default Africa/Dar_es_Salaam
is_active	default true

Per-entity currency and timezone matter — the same installation can run pharmacies in different countries, and each one's reports come out in its own currency.

2. employees — the person

COLUMN	NOTES
auth_user_id	→ auth.users, UNIQUE — one login ↔ one employee
full_name, username, email	text
role	master_admin / entity_admin / inventory_user / sales_user
branch_id	home entity. NULL only for master admins
permission_overrides	jsonb {"<permission>": true false}, default {}
max_discount_percent	this person's discount ceiling
approval_limit	numeric
status	active / disabled

Two things to point out:

- **Login and employee are separate.** Supabase Auth holds the password; employees holds the business identity. auth_user_id joins them. A disabled employee can still authenticate but gets nothing — the DAL rejects any status other than active.
- max_discount_percent is per-person, not per-role. So a senior cashier can be trusted with 15% while a new one gets 5%, without inventing new roles.

3. The four roles

ROLE	IN ONE LINE
master_admin	Owns the whole installation. Sees all entities. The only role that can create entities.
entity_admin	Owns one pharmacy completely. Cannot create entities or reach other ones.
inventory_user	Buys and stocks. Products, suppliers, inward, stock-out. Cannot sell.
sales_user	Dispenses at the counter. Cannot see purchase cost or profit.

THE TWO COMMERCIALY IMPORTANT SEPARATIONS

The person at the counter **cannot see what you paid** for the medicine or what you make on it.

The person who buys stock **cannot ring up sales**, so nobody controls both sides of a transaction.

4. Where permissions live

Three pieces, and the layering is the point.

role_permissions — the template

(role, permission) composite PK, **53 rows** = 4 roles × their permissions. Seeded in migration 0006, and it is the *runtime* source of truth — not a constant in the code.

employees.permission_overrides — the exceptions

Per-person adjustments on top. {"apply_discount": true} grants it; {"view_profit": false} revokes it. So you rarely need to invent a new role: you take the nearest one and adjust.

permission_catalog — what may be adjusted at all

permission PK, overridable bool, all 21 permissions. Three are overridable = false:

```
manage_users · manage_entities · access_multiple_entities
```

WHY THIS TABLE EXISTS

These three are role-only, never grantable by override. Without this, anyone holding manage_users could hand themselves — or a colleague — master-level reach by writing one JSON key. That is the classic privilege-escalation hole, closed by data rather than by code review.

employee_entities — extra access

(employee_id, branch_id) composite PK. Grants a person access beyond their home entity. Master admins need no rows here; they see everything by role.

5. The 21 permissions

GROUP	PERMISSIONS
Products	view_products create_products edit_products import_products
Inventory	view_inventory adjust_inventory create_stock_inward create_stock_outward approve_stock_outward
Purchasing	view_purchase_cost manage_suppliers
Sales	create_sales apply_discount cancel_sales
Management	view_profit view_management_reports generate_exports
Administration	manage_users * manage_entities * manage_settings access_multiple_entities *

* the three non-overridable, master-tier permissions.

6. The role matrix

The four rows worth reading aloud are in bold.

PERMISSION	MASTER	ENTITY_ADMIN	INVENTORY	SALES
view_products	✓	✓	✓	✓
create / edit / import products	✓	✓	✓	—
view_inventory	✓	✓	✓	✓
adjust_inventory	✓	✓	—	—
create_stock_inward	✓	✓	✓	—
create_stock_outward	✓	✓	✓	—
approve_stock_outward	✓	✓	—	—
view_purchase_cost	✓	✓	✓	—
manage_suppliers	✓	✓	✓	—
create_sales	✓	✓	—	✓
apply_discount	✓	✓	—	✓
cancel_sales	✓	✓	—	—
view_profit	✓	✓	—	—
view_management_reports	✓	✓	—	—
manage_users	✓	✓	—	—
manage_entities	✓	—	—	—
manage_settings	✓	✓	—	—
access_multiple_entities	✓	—	—	—
generate_exports	✓	✓	—	—

Note `create_stock_outward` versus `approve_stock_outward`: an inventory user can *raise* a write-off, but only an admin can *approve* it. Stock does not leave on one person's say-so.

7. How a permission check resolves

The rule: role template first, then overrides on top — *but* overrides are ignored for the three non-overridable permissions.

That rule is implemented **twice, deliberately**:

- In TypeScript — `resolvePermissions()` in `src/lib/permissions.ts` , which decides what the UI renders and what actions accept.
- In SQL — `has_perm(text)` , which every one of the 97 RLS policies calls.

They are not duplicated by accident. The TypeScript copy gives a clean UI and a friendly error message; the SQL copy is the one that actually protects the data. If someone bypassed the app entirely and hit the API directly, `has_perm` still runs.

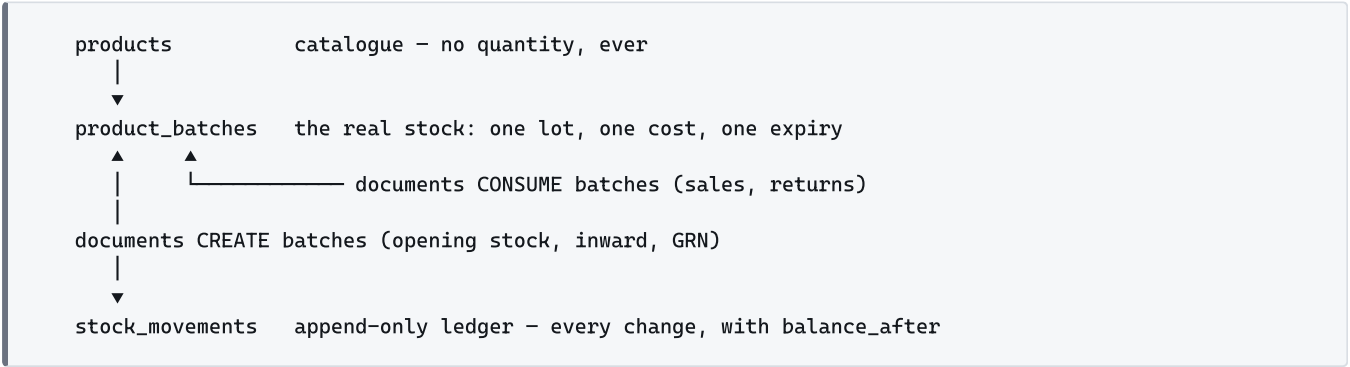
And there is a third guard: the `employees_guard` trigger **rejects** a non-overridable override at write time, on every path — *including the service-role key that bypasses RLS*. So a bad override cannot be stored, cannot be read, and would not work if it somehow existed.

ONE-SENTENCE VERSION

Four roles give you the defaults, per-person overrides handle the exceptions, three master-tier permissions are locked out of the override mechanism entirely, and the same resolution rule is enforced independently in the app and in the database.

Products, batches, documents, the ledger

The operational heart of the system. Everything here exists to make one claim true — and the claim is at the end of this step.



1. products — the catalogue

COLUMN	NOTES
sku	UNIQUE (branch_id, sku) — per entity, <i>not</i> global
name, generic_name, strength, form, unit, barcode, manufacturer	text
category_id, supplier_id	uuid FK
buy_price	LOCKED SELECT revoked from authenticated
sell_price	numeric
pricing_method	fixed cost_plus_margin
margin_percent	used when cost_plus_margin
max_discount_percent	per-product ceiling
reorder_level / restock_target	when to reorder / reorder up to
status	active discontinued quarantined

Three things to flag:

- **No quantity column.** Deliberate. Stock is never on the product.
- **SKUs are unique per entity.** Two pharmacies can both use PARA-500 for their own products without collision.
- **buy_price has SELECT revoked at the database level.** Not hidden by a query — *revoked*. A sales user's connection cannot read that column at all.

TWO DISCOUNT CEILINGS

`products.max_discount_percent` (this medicine) and `employees.max_discount_percent` (this cashier). A sale must satisfy **both** — checked inside `erp_complete_sale`, not in the browser.

`product_price_history` logs every change to `buy_price`, `sell_price`, `margin_percent`, `pricing_method` and `max_discount_percent` — with old value, new value, who, and why. Rows where `field = 'buy_price'` require `view_purchase_cost` to read.

2. `product_batches` — the real stock

COLUMN	NOTES
<code>product_id</code> , <code>branch_id</code>	uuid NOT NULL
<code>batch_number</code>	text NOT NULL
<code>supplier_id</code>	uuid FK
<code>quantity_received</code>	as delivered
<code>quantity_available</code>	LOCKED UPDATE revoked — ledger-derived
<code>unit_cost</code>	LOCKED SELECT revoked from <code>authenticated</code>
<code>expiry_date</code> , <code>storage_location</code>	date, text
<code>status</code>	<code>active</code> <code>quarantined</code> <code>damaged</code> <code>expired</code> <code>negative</code>
<code>source_type</code> / <code>source_id</code>	the document that created this batch

THE MOST IMPORTANT REVOCATION IN THE SCHEMA

`quantity_available` has **UPDATE revoked**. Nobody — no user, no app code, no admin screen — can write a number into that column. It only moves when a ledger row is posted. That single revocation is what makes “the ledger is the truth” a structural fact rather than a convention.

`source_type` / `source_id` give full traceability: any batch on the shelf points back to the document that brought it in. Pharmacy regulators ask for exactly this.

3. Stock IN — three doors, all draft → confirm

A. `opening_stock_entries` → `opening_stock_items`

Day-one stock when a pharmacy joins the system. Line carries `product_id`, `batch_number`, `expiry_date`, `quantity` (CHECK > 0), `unit_cost`, optional `sell_price`.

Confirm: `erp_confirm_opening_stock(id)` → one batch per line, posts `opening_stock` movements, locks the entry.

B. stock_inwards → stock_inward_items

Routine goods receipt. Four `inward_type` values:

```
purchase_from_parent · purchase_from_external · foc_or_sample · replacement_in
```

Each line has both `quantity` and `free_quantity` — paid units and free units on the same line, with `CHECK (quantity + free_quantity > 0)`.

Confirm: `erp_confirm_stock_inward(id)`. **Paid units post as `purchase` ; free units post as a separate `foc` movement.**

A DIFFERENTIATOR WORTH NAMING

Free goods stay identifiable in the ledger forever — so you can answer “how much of our margin came from free stock?” That is a real pharmacy-distribution question and most systems cannot answer it.

Also: `UNIQUE (branch, supplier, invoice_number)` while not cancelled — the same supplier invoice cannot be received twice.

C. purchase_orders → received_orders

The formal PO path.

```
PO status:  draft → pending_approval → approved → partially_received
            → received | cancelled
```

The GRN (`received_orders`) carries `quantity_ordered` , `quantity_received` and `damaged_qty` per line, with status `partial` | `complete` | `variance` .

Receive: `erp_receive_purchase_order(po_id, supplier_invoice, lines)`.

4. Stock OUT

sales → sale_items

COLUMN	NOTES
<code>invoice_number</code>	UNIQUE
<code>customer_id</code> , <code>cashier_id</code>	uuid FK
<code>payment_method</code>	<code>Cash</code> <code>Bank</code> <code>M-Pesa</code> <code>Selcom</code> <code>Credit</code>
<code>subtotal</code> , <code>discount</code> , <code>tax</code> , <code>total</code>	always derived server-side
<code>status</code>	<code>completed</code> <code>returned</code> <code>reversed</code>
<code>request_key</code>	UNIQUE (branch_id, request_key) — idempotency

- **Money is never trusted from the browser.** The client sends product ids and quantities; `erp_complete_sale` computes every figure from the database. A tampered request cannot change a price.
- **`request_key` is a double-click guard.** Same key twice = the original sale returned, not a second one. Enforced by a unique index, so it holds even under a race.
- **`sale_items` is immutable** — no UPDATE or DELETE policy exists.

Cancelling a sale calls `erp_reverse_sale(sale_id, reason)`, which **never deletes**. It posts `sale_reversal` movements and stamps `reversed_by` / `reversed_at` / `reversal_reason`. Nothing disappears from the record.

`returns` — one table for every stock-out document

TYPE	REAL-WORLD EVENT	STOCK EFFECT
<code>customer</code>	customer brings it back	+ back in
<code>supplier</code>	send it back to the supplier	– out
<code>damaged</code>	broken	– out
<code>expired</code>	past expiry	– out
<code>employee_consumption</code>	staff used it	– out

Status: `pending` → `approved` | `review` | `rejected` | `completed`. Approve with `erp_approve_stock_out(return_id)` — which needs `approve_stock_outward`, a permission inventory users do not have.

Why one table: all five are “stock leaves or returns with a documented reason and a named approver.” So they share one approval flow, one audit trail, one screen — and the `type` column keeps them distinct in the ledger.

`stock_transfers`

Move a batch between entities. `erp_transfer_stock(batch_id, to_branch, quantity)` posts **both** `transfer_out` and `transfer_in` legs atomically. Never one without the other.

5. `stock_movements` — the ledger

COLUMN	NOTES
<code>product_id</code> , <code>batch_id</code> , <code>branch_id</code>	uuid
<code>movement_type</code>	13 types
<code>quantity_delta</code>	signed
<code>balance_after</code>	running balance for that batch
<code>reference_type</code> / <code>reference_id</code> / <code>reference_number</code>	back-link to the source document
<code>reason</code> , <code>created_by</code> , <code>created_at</code>	

LOCKED DOWN

SELECT only. No INSERT / UPDATE / DELETE policy *and* the privileges are revoked from `authenticated` . There is exactly **one writer**: `fn_post_movement` , which is revoked from every client role and callable only from inside an `erp_*` RPC.

So the ledger cannot be written by the app, by a user, or by an admin screen. Only by a database function reached through one of the nine transactional RPCs.

The stock equation

```
closing stock = opening_stock
                + purchase + foc + replacement_in + transfer_in
                - sale - employee_consumption - expiry - damage
                - supplier_return - transfer_out
                + sale_reversal
                ± stock_correction
```

And it is tested. The suite asserts the ledger sums exactly to `product_batches.quantity_available` , and that `balance_after` forms an unbroken chain. If anything ever broke the invariant, the tests fail.

Movement types

PHASE 1 VOCABULARY	LEGACY (PRE-EXISTING ROWS)
<code>opening_stock</code> <code>purchase</code> <code>foc</code> <code>replacement_in</code> <code>sale</code>	<code>purchase_receipt</code> <code>adjustment</code> <code>return</code>
<code>employee_consumption</code> <code>expiry</code> <code>damage</code> <code>supplier_return</code>	<code>disposal</code> <code>count_correction</code>
<code>sale_reversal</code> <code>stock_correction</code> <code>transfer_in</code> <code>transfer_out</code>	

6. Supporting tables

TABLE	PURPOSE
customers	name, phone, segment, loyalty_points, credit_balance
expenses	reference UNIQUE, category_id, vendor, amount, status
audit_logs	employee_id, action, module, previous_value, new_value, reason
login_history	device, ip_address, session_ref, status
notifications	branch_id NULL = global
settings	branch_id NULL = global, UNIQUE (branch_id, key)
approval_tasks	type, reference_id, requested_by, amount, status
stock_counts / stock_count_items	Physical count sheets; variance is a generated column
document_embeddings	AI retrieval — embedding vector, entity-scoped
product_imports / draft_products	Import staging — see below

Imports never create sellable products directly

Rows land in `draft_products` as `pending` and become real products only when a user with `create_products` confirms them. `product_imports` carries **UNIQUE (branch_id, kind, file_hash)** — the same file cannot be imported twice into one entity, whatever it is renamed to.

Limits: 5 MB, 5 000 rows, `.xlsx` / `.csv` only. Rows are re-validated and sell price recomputed at commit, because the payload round-trips through the browser.

Document numbering

Six sequences — `doc_seq_sale`, `doc_seq_inward`, `doc_seq_opening`, `doc_seq_return`, `doc_seq_correction`, `doc_seq_import` — issued by `next_doc_number(prefix, seq)` → `INV-2026-000123`. Collision-proof under concurrency.

THE CLAIM THIS ALL BUYS YOU

For any unit of stock, you can answer: **where did it come from, what did it cost, when does it expire, who touched it, and where did it go** — and the numbers are arithmetically guaranteed to reconcile.

That is the sentence to say out loud. Everything in this step exists to make it true.

STEP 5

The security model

Five enforcement layers, every one of them inside PostgreSQL. The application layer is for user experience, not for defence.

Start with the threat model

SAY THIS FIRST

The browser holds the anon key and the user's JWT. So nothing in the client is a security control.

It is the sentence that makes the rest make sense. Anyone with devtools can read the anon key out of the page and hit the Supabase REST API directly — skipping the login page, the React code, the server actions, all of it.

So the question is never “does the UI hide it?” It is: **what happens when someone talks straight to the database?**

Browser (hostile) → Server Actions
→ Direct PostgREST

┌
└
└
↓

← the real threat model

LAYER 1 Column privileges buy_price / unit_cost not selectable
LAYER 2 RLS · 97 policies has_entity_access() AND has_perm()
LAYER 3 Permission resolution role + overrides + catalogue
LAYER 4 Transactional RPCs permission → entity → consistency → lock
TRIGGERS fire on every path, service role included

Layer 1 — Column privileges

The strongest thing available, and used in exactly three places.

TARGET	REVOKED
products.buy_price	SELECT from authenticated
product_batches.unit_cost	SELECT from authenticated
product_batches.quantity_available	UPDATE from authenticated

This is stronger than a policy. A policy filters rows; this removes the column from the role's reach. A sales user asking for buy_price does not get null — they get an error, because that column is not theirs to name.

Cost is reachable only through two views:

```
create view product_costs with (security_invoker = false) as
select p.id as product_id, p.branch_id, p.buy_price
from products p
where has_entity_access(p.branch_id)
      and has_perm('view_purchase_cost');
```

`batch_costs` is the same shape for `unit_cost`. **Without the permission you get zero rows** — not a nulled column, not a masked value. The data does not exist for you.

MAINTENANCE NOTE

Column privileges are role-wide, so the pattern is *revoke table SELECT, then grant back the permitted columns*.

`fn_grant_readable_columns(table, exclude[])` does that, and must be re-run whenever a column is added to either table.

Layer 2 — Row-level security

Enabled on 36 / 36 tables. 97 policies. No exceptions.

Every policy composes the same two functions:

```
has_entity_access(branch_id)  AND  has_perm('view_inventory')
      ↑                               ↑
    which pharmacy?          which capability?
```

Two tables are SELECT-only by design: `stock_movements` (the ledger) and `document_embeddings`.

`has_entity_access(uuid)` is true for a master admin, or when the branch is the employee's own, or when it is granted via `employee_entities`. And one detail worth stating: `has_entity_access(NULL)` returns **false**. Null is not a wildcard. A row with no entity is invisible to everyone rather than visible to everyone — that was a real finding, fixed in migration `0017`.

Layer 3 — Permission resolution

`has_perm(text)` is the SQL half of what Step 3 described:

```
select case
  when e.permission_overrides ? p
    and coalesce((select pc.overridable from permission_catalog pc
                  where pc.permission = p), false)
  then (e.permission_overrides ->> p)::boolean
  else exists (select 1 from role_permissions rp
              where rp.role = e.role and rp.permission = p)
end
from employees e
where e.auth_user_id = auth.uid() and e.status = 'active'
```

Read it out loud: *use the override only if the catalogue says this permission is overridable; otherwise fall back to the role template.* A planted `{"manage_entities": true}` is **ignored at read time**, not just rejected at write time.

Note `e.status = 'active'` — a disabled employee resolves to no permissions at all, at the database level.

Function grants

Locked down in migration `0009`:

ACCESS	FUNCTIONS
Callable by <code>authenticated</code>	<code>has_perm</code> <code>has_entity_access</code> <code>is_master</code> <code>auth_employee_id</code> <code>next_doc_number</code> <code>fn_path_entity</code>
Revoked from every client role	<code>fn_post_movement</code> <code>fn_current_employee</code> <code>fn_employees_guard</code> <code>fn_protect_last_master</code> <code>fn_grant_readable_columns</code>

`fn_post_movement` being in that second list is what makes the ledger unforgeable.

Layer 4 — Transactional RPCs

Nine functions. All `SECURITY DEFINER`, all with pinned `search_path`, all granted to `authenticated` only, all **commit whole or roll back whole**.

FUNCTION	DOES
<code>erp_complete_sale(p jsonb)</code>	Locks batches, validates stock / entity / discount, prices from the DB , writes sale + items + ledger. Idempotent via <code>request_key</code> .
<code>erp_reverse_sale(id, reason)</code>	Posts <code>sale_reversal</code> movements; never deletes
<code>erp_confirm_opening_stock(id)</code>	Batches + <code>opening_stock</code> movements, locks the entry
<code>erp_confirm_stock_inward(id)</code>	Batches; paid and FOC movements posted separately
<code>erp_approve_stock_out(id)</code>	Validates batch / product belong to the document's entity, posts the movement
<code>erp_stock_correction(batch, qty, reason)</code>	Privileged quantity fix; reason mandatory
<code>erp_transfer_stock(batch, to, qty)</code>	Both ledger legs atomically
<code>erp_receive_purchase_order(po, invoice, lines)</code>	GRN + batches + ledger, rolls the PO forward
<code>erp_commit_product_import(...)</code>	Import record + drafts + audit in one transaction

Every one validates in the same order

permission → entity access → document status → branch consistency
→ business rules → row locks → ledger → commit

Branch consistency (migration `0013`) is the subtle one: it checks that the batch, product and supplier all belong to the *document's* entity. Without it you could reference your own entity's document while pointing at another entity's batch — passing every individual check while still crossing the boundary.

Row locks — `SELECT ... FOR UPDATE` — mean two cashiers selling the last box at the same instant cannot both succeed. And `fn_post_movement` refuses to take stock negative, under that lock.

The triggers — the layer that ignores the service-role key

`employees_guard` (BEFORE INSERT OR UPDATE)

1. Rejects non-overridable keys, unknown keys and non-boolean values in `permission_overrides` — **on every path, including the service role**
2. **Nobody may change privilege columns on their own row**, whatever permissions they hold (`role` , `permission_overrides` , `max_discount_percent` , `branch_id` , `approval_limit` , `status`)
3. Non-master privilege writes must satisfy `has_entity_access` on both the old *and* the new branch — so you cannot move someone into your entity to gain control of them, or out of it
4. Only a master admin may grant `master_admin` or change `auth_user_id`

`employees_protect_last_master` (BEFORE UPDATE)

The last active master admin cannot be disabled or demoted. No locking yourself out of your own installation.

WHY “SERVICE ROLE INCLUDED” MATTERS

RLS is *bypassed* by the service-role key. **Triggers are not.** So these rules hold even for code paths that hold the god key.

Storage

Buckets `product-images` and `stock-documents` : **private**, 10 MB cap, MIME allowlist (PNG / JPEG / WebP / GIF / PDF). Read, write and delete are confined to a `<branch_id>/...` path prefix the caller can actually access — entity isolation extends to files.

What is enforced where

This is the one to put on a slide.

RULE	ENFORCED BY
Entity A cannot read / write Entity B	RLS on every table
Sales user cannot see purchase cost	Column privileges + gated views
Ledger cannot be forged	INSERT revoked + RPC-only writes
Stock cannot go negative	<code>fn_post_movement</code> under row lock
Discounts within user <i>and</i> product ceilings	<code>erp_complete_sale</code>
No self-privilege escalation	<code>employees_guard</code> trigger
Master permissions not grantable by override	<code>permission_catalog</code> + trigger + <code>has_perm</code>
Documents cannot be confirmed twice	<code>SELECT ... FOR UPDATE</code> + status check
Duplicate sale submission	<code>UNIQUE (branch_id, request_key)</code>
Duplicate file import	<code>UNIQUE (branch_id, kind, file_hash)</code>

Migration order — and why it is load-bearing

`0002` installs permissive bootstrap policies that `0007` replaces with the real per-entity ones. **Applying them out of order leaves the database open.**

#	FILE	PURPOSE
0001	<code>schema.sql</code>	Core tables
0002	<code>rls.sql</code>	Bootstrap RLS (superseded by 0007)
0003	<code>seed.sql</code>	Demo catalogue
0004	<code>seed_admin_employee.sql</code>	First admin ↔ employee link
0005	<code>ai_embeddings.sql</code>	Vector search
0006	<code>erp_foundation.sql</code>	Entities, roles, permissions, product master, stock documents
0007	<code>erp_rls.sql</code>	Per-entity + per-permission RLS
0008	<code>erp_stock_rpcs.sql</code>	Transactional stock RPCs
0009	<code>erp_function_grants.sql</code>	Revoke PUBLIC / anon EXECUTE
0010	<code>erp_import_rpc.sql</code>	Atomic import commit
0011	<code>permission_override_whitelist.sql</code>	<code>permission_catalog</code>
0012	<code>employee_privilege_guard.sql</code>	Anti-escalation trigger
0013	<code>rpc_branch_consistency.sql</code>	Branch consistency in every RPC
0014	<code>ledger_write_lockdown.sql</code>	Ledger INSERT revoked
0015	<code>cost_column_privileges.sql</code>	Cost views
0016	<code>cost_column_privileges_fix.sql</code>	Column privilege lockdown
0017	<code>med_hardening.sql</code>	Null-entity, audit attribution, idempotency, storage

Apply: `supabase db push`. **Fresh empty database:** `npm run db:bundle`, then paste `supabase/migrations/run_all.sql`.

How it is verified

```

npx tsc --noEmit    exit 0
npm run lint        exit 0
npm test            45 unit tests
npm run test:db     103 tests against real RLS with real signed-in users
npm run build       exit 0

```

THE CREDIBILITY LINE

*148 tests, roughly 74 of them adversarial — they sign in with the **anon key** and attack through PostgREST, the same path an attacker with devtools would take. The security claims are not asserted; they are attacked, and they hold.*

Be ready for the honest question

If someone asks “so is it secure?”, the answer is: *the enforcement model is sound and tested; there are known open items.*

ITEM	STATUS
Credential rotation (service-role, <code>sb_secret</code> , anon, OpenAI, exposed admin password)	Outstanding — highest risk
Dedicated test Supabase project	Guard shipped; project not created
Browser QA (XSS / CSRF / session)	Not yet performed
Dependencies	3 high, 2 moderate, 0 critical — not upgraded

Volunteering these is stronger than being caught by them. The first is a genuine action item — those credentials were exposed during development and need rotating before this goes anywhere near production. If the audience includes anyone who could authorise that, raise it.

STEP 6

Presentation script

A timed running order, the five-move demo, prepared answers to the hard questions, and what to do the night before.

The 25-minute structure

MIN	SECTION	THE ONE THING THEY SHOULD TAKE AWAY
0–3	What it is	One installation, many independent pharmacies, isolated by the database
3–8	The four ideas	Batches not products · ledger is truth · draft→confirm · entity is the boundary
8–12	Who can do what	4 roles, 21 permissions, and the two commercial separations
12–20	Live demo	The isolation and cost-hiding are real, not cosmetic
20–24	How it is proven	148 tests, 74 of them attacking through the anon key
24–25	Open items	Credential rotation is the outstanding risk

If cut to 10 minutes: do the four ideas and the demo. Drop the rest.

Your opening line

SAY THIS

"This is one pharmacy ERP installation that runs many independent pharmacies. Each one is called an Entity. No entity can see another's stock, costs, sales or staff — and that is enforced by PostgreSQL, not by our application code."

Then immediately: *"Let me show you why that distinction matters."*

The demo — five moves, in this order

The QA data gives you **Test Pharmacy A** (QA-A , Dar es Salaam) and **Test Pharmacy B** (QA-B , Arusha), with six accounts across all four roles, plus products, suppliers, customers, confirmed opening stock, an inward with free goods, completed sales, approved stock-outs and expenses.

BEFORE YOU START

Logins are in `.qa-credentials.local.txt` — gitignored, never committed. **Point the demo at your QA project, not production.**

Move 1 — Entity isolation (the money shot)

Sign in as **Entity Admin A**. Show the inventory list. Note there is no entity switcher — they have one pharmacy.

Sign in as **Entity Admin B**. Completely different products, suppliers, sales.

Then say: *"Same database. Same tables. Same API. The rows simply do not exist for the other user."*

Move 2 — Cost concealment

Sign in as the **sales user**. Open a product. **No purchase cost. No profit figures.**

Now the line that lands: *"This is not the UI hiding a column. The `SELECT` privilege on `buy_price` is revoked for that role — the column is not readable by that connection at all. Even hitting the API directly returns nothing."*

Switch to the **inventory user** — cost is there. Same screen, different truth.

Move 3 — Draft → confirm

Open a stock inward. Show it as a `draft`, edit a line. Point out: **inventory has not moved.**

Confirm it. Now show the batches that appeared and the ledger rows behind them.

"One database transaction. Batches, ledger, document lock — all or nothing. There is no code path that does half of it."

Move 4 — The ledger

Open the stock ledger for one batch. Show the signed movements and the running `balance_after`.

"Append-only. `INSERT` is revoked — even our own application cannot write here. Only one database function can, and it is unreachable from any client. The ledger sums exactly to the batch quantity, and a test asserts that."

If you show one thing from the free-goods inward: point out the paid units and the FOC units posted as **separate movement types**. Most systems cannot tell you how much of your margin came from free stock. This one can.

Move 5 — The master view

Sign in as **master admin**. Show the entity switcher, including "all entities" consolidated mode.

"This is the only role that sees across pharmacies, and the only role that can create one."

Questions you should expect, and the answers

Q What stops a developer, or someone with devtools, bypassing the app?

Nothing needs to stop them — the app is not the control. The browser already holds the anon key. Every rule lives in Postgres: 97 RLS policies, revoked column privileges, and nine transactional functions. And 74 of our tests are written from exactly that attacker's position, signing in with the anon key and hitting the REST API directly.

Q Could an entity admin promote themselves to master admin?

No, in three independent ways. Master-tier permissions are marked non-overridable in `permission_catalog`. A trigger rejects the write. And `has_perm` ignores such an override even if it somehow existed. On top of that, nobody can edit privilege columns on their own row, whatever their permissions.

Q What if two cashiers sell the last box at the same time?

One succeeds, one gets an error. The sale function takes a row lock on the batch, and the ledger writer refuses to go negative under that lock.

Q What if a cashier double-clicks Complete Sale?

One sale. Each submission carries a request key with a unique constraint on `(branch_id, request_key)` — the second attempt returns the original sale rather than creating a new one. Enforced by a unique index, so it holds under a race, not just under a check.

Q Can someone delete a sale to hide it?

No. `sale_items` has no UPDATE or DELETE policy at all. Cancelling posts reversal movements and stamps who reversed it, when, and why. Nothing is ever removed from the record.

Q Why one `returns` table for five different things?

Because customer returns, supplier returns, damage, expiry and staff consumption are all the same shape: stock moves with a documented reason and a named approver. One table means one approval flow, one audit trail, one screen — and the `type` column keeps them distinct in the ledger.

Q Is it production-ready?

The enforcement model is sound and tested. Four things are outstanding: credentials exposed during development need rotating — that is the highest risk and it is a decision I need from you; a dedicated test project needs creating; browser-level QA has not been done; and there are 3 high-severity dependency advisories, no critical.

Answer that last one straight. Volunteering it reads as competence; getting caught by it does not.

Numbers to have on hand

36 tables · 2 views · 97 RLS policies · 9 transactional RPCs · 17 migrations
21 permissions · 4 roles · 53 role-permission rows
24 screens · 19 server actions · 25 data loaders
148 tests (45 unit + 103 database), ~74 adversarial
Next.js 16 · React 19 · PostgreSQL 15

Commands, if you want to run anything live

```
npm run dev          # http://localhost:3000
npm test             # 45 unit tests – fast, safe, no DB
npm run test:db      # 103 RLS/RPC tests against a real project
npx tsc --noEmit ; npm run lint
```

`npm test` is the safe one to run in front of people — a few seconds, no database, and green. Save `test:db` for beforehand; it writes to whatever project `.env.local` points at.

Re-seeding the demo data — do this **the night before**, not live:

```
npx tsx scripts/setup-qa-accounts.ts    # entities + 6 accounts
npx tsx scripts/seed-qa-data.ts        # catalogue, stock, sales, expenses
```

Pre-flight checklist

- ☐ Run the two seed scripts; confirm both pharmacies have data.
- ☐ Sign in as **all four** roles once and confirm each looks how you will describe it. This is the step people skip and regret.
- ☐ Run `npm test` so you know it is green.
- ☐ Have `docs/SCHEMA.md` open in a tab — your backup for any detail question.
- ☐ Confirm the demo is pointed at the QA project, not production.

IF YOU REMEMBER ONLY ONE PARAGRAPH

Each pharmacy is an isolated entity. Stock lives on batches with their own cost and expiry, never on products. Every stock change appends to a ledger that the application itself cannot write to, and the ledger provably reconciles to on-hand quantity. Documents do not move stock until they are confirmed, and confirming is one all-or-nothing database transaction. Purchase cost is revoked at the column level for counter staff, and no user can raise their own privileges. All of it is enforced in PostgreSQL, and 74 tests attack it the way an attacker actually would.
